Decentralized Voting System Using Ethereum Blockchain & MetaMask

> A secure, transparent, and tamper-proof online voting application built on the Ethereum blockchain network.

Team Members

Name	App ID
Aaryan Sah	2306528
Agniv Banerjee	2304952
Aum Tripathi	2390080
Github repo link:	https://github.com/aaryansah7/E-Voting-System-using-Blockchain-Ganache-MetaMask-.git

Project Domain: Blockchain Technology & Web Development

Project Description

This project implements a **Decentralized Voting System** on the Ethereum blockchain. It allows eligible voters to cast their votes and view results without the need for any central authority or intermediary. Votes are recorded as immutable transactions on the blockchain, making them impossible to manipulate or alter. Smart contracts automate the entire voting process, ensuring transparency, security, and cost-effectiveness.

Problem Statement

Traditional voting systems — both paper-based and electronic — suffer from a range of critical issues including vulnerability to fraud, lack of transparency, slow result processing, high operational costs, and limited voter accessibility. Centralized electronic systems introduce single points of failure and are susceptible to hacking or manipulation by controlling authorities. There is a need for a reliable, trustless, and publicly verifiable election infrastructure.

Objectives

- Provide a **secure and tamper-proof** platform for conducting elections using blockchain immutability.
- Ensure complete **transparency** — any participant can verify votes and results in real time.
- Improve **accessibility** by enabling voters to cast votes from anywhere with an internet connection.
- Increase **efficiency** by automating the voting process through smart contracts, reducing cost and manual effort.
- Build **trust** in the democratic process through a verifiable, decentralized mechanism.

Key Features

- **JWT-based Authentication** — Secure login for voters and admins using token-based authorization.
- **Smart Contract Voting** — Votes are cast and recorded directly on the Ethereum blockchain via Solidity smart contracts.
- **Role-based Access** — Separate dashboards for Voters and Administrators.
- **Admin-defined Voting Period** — Admins set start/end dates; voting is only active during this window.
- **Real-time Vote Count** — Live vote tallies are visible on the voting page.
- **Remote Voting** — No physical presence required; voters can participate from anywhere.
- **One Voter, One Vote** — Smart contract enforces that each wallet address can only vote once.

Quick Navigation

Page	Description
Introduction	Background, motivation, and existing vs proposed system
Objectives and Scope	Detailed objectives, use cases, and expected outcomes
Literature Survey	Research papers reviewed and comparison of tools
System Architecture	Architecture diagram and component breakdown
Technologies Used	Languages, frameworks, and tools
Methodology	Step-by-step working of the system
Implementation	Setup guide, folder structure, and code snippets

Page	Description
Results	Test cases, output screenshots, and observations
Challenges	Problems faced during development
Future Scope	Planned enhancements and research opportunities
Conclusion	Summary and key learnings

Introduction

1.1 Background — What is Blockchain?

Blockchain is a **distributed digital ledger technology** that allows participants in a network to share and validate transactions in a secure and transparent manner without the need for intermediaries. The technology is designed to be decentralized — data is stored across a network of computers rather than a single central database, making it extremely difficult to hack or manipulate.

Blockchain works by creating **blocks of data linked together in a chain**. Each block contains a unique cryptographic hash generated from its contents. This hash links the block to the previous one, forming a tamper-evident chain. Once a block is added, it cannot be altered or deleted without the consensus of the entire network — making the system **immutable and transparent**.

Originally popularized by Bitcoin, blockchain has since been applied across industries including:

- Finance and cryptocurrency
- Supply chain management
- Healthcare records
- **Electronic Voting Systems** (the focus of this project)

1.2 Motivation — Why Decentralized Voting?

A decentralized voting system built on the Ethereum blockchain has the potential to revolutionize the way elections are conducted. By leveraging the security, transparency, and immutability of blockchain, it can eliminate many risks associated with traditional voting:

- Each voter has a **unique digital identity**; their vote is recorded on-chain and cannot be altered.
- There is **no need for government agencies or third-party intermediaries** to oversee the election.
- Voters can cast ballots **from anywhere in the world**, leading to higher voter turnout and greater democratic inclusion.

1.3 Existing Systems

The current voting infrastructure typically falls into two categories:

Paper-Based Voting

Voters physically visit a designated polling location to cast votes on paper ballots. These ballots are manually counted and recorded.

Electronic Voting Machines (EVMs) / Online Systems

Some countries have adopted electronic or internet-based voting systems. However, these have faced significant criticism due to security vulnerabilities and lack of auditability.

1.4 Disadvantages of Existing Systems

#	Limitation	Description
1	**Lack of Transparency**	Voters have no way to verify if their vote was counted correctly; observers cannot audit the process independently.
2	**Vulnerability to Fraud**	Paper ballots and voting machines can be tampered with, hacked, or manipulated. No paper trail means no reliable audit.
3	**Slow Results**	Manual ballot counting is time-consuming and delays the announcement of election results.
4	**High Cost**	Running traditional elections requires hiring poll workers, renting polling places, and purchasing voting equipment.
5	**Centralization**	Centralized systems are controlled by a small number of authorities, creating potential for abuse of power or systemic manipulation.
6	**Limited Accessibility**	Voters must travel to specific polling places, which is difficult or impossible for people with disabilities or those in remote locations — resulting in voter disenfranchisement.

1.5 Proposed Solution

The proposed **Decentralized Voting System using Ethereum Blockchain** addresses all of the above limitations:

- **Smart Contracts** on the Ethereum network automate the voting process — no manual intervention required.
- Every vote is a **blockchain transaction** — immutable, verifiable, and publicly auditable.
- **Role-based access control** separates Admin (election management) and Voter (ballot casting) functionalities.
- **JWT authentication** with a MySQL database ensures only registered voters can access the system.
- **Real-time vote tallying** is built into the voting page — results are updated live as votes are cast.
- The system is **cost-effective** — no physical infrastructure, no intermediaries.

> This approach provides a reliable, transparent, and accessible solution for conducting trustworthy and fair elections in the digital age.

Objectives and Scope

Project Objectives

1. Security

Provide a secure platform for conducting elections by eliminating the possibility of vote tampering. The Ethereum blockchain's cryptographic properties and the use of smart contracts ensure that no single entity can alter the recorded votes or manipulate the results.

2. Transparency

Allow voters to view the entire voting process in real time — including the live vote count and final results. All transactions are publicly recorded on the blockchain, ensuring end-to-end verifiability.

3. Accessibility

Eliminate the need for physical presence at a polling station. Any registered voter with an internet connection and valid credentials can participate from any location in the world, increasing voter turnout and inclusivity.

4. Efficiency

Reduce the time and resources required to conduct elections by automating the process through smart contracts. This removes the need for manual counting, poll workers, and physical infrastructure — significantly cutting costs.

5. Trust

Increase public trust in the voting process by providing a tamper-proof, transparent mechanism for recording and tallying votes. The decentralized nature of the system means no single party controls the outcome.

Scope of the Project

The scope of this project covers the design and development of a full-stack decentralized voting application. It includes:

- A **Solidity smart contract** deployed on a local Ethereum network (Ganache) that handles candidate registration, date management, and vote casting.
- A **Python FastAPI backend** connected to a MySQL database for voter authentication and JWT token management.
- A **JavaScript frontend** using Web3.js and Truffle to interact with the deployed smart contract via MetaMask.
- A **two-role system**: Admin (manages election setup) and Voter (casts votes).
- **Testing** using unit, functional, and black/white-box testing methodologies.

> **Note:** The current implementation runs on a local Ganache blockchain (testnet). Deployment to the Ethereum mainnet or a public testnet (e.g., Sepolia) is considered a future enhancement.

Use Cases

Use Case 1: Voter Login

- **Actor:** Voter
- **Description:** A registered voter enters their Voter ID and password. The backend validates credentials against the MySQL database and returns a JWT token. The voter is redirected to the Voting Page.
- **Outcome:** Voter gains access to the ballot.

Use Case 2: Admin Login

- **Actor:** Admin
- **Description:** An administrator logs in with admin credentials. Upon successful authentication, they are redirected to the Admin Panel.
- **Outcome:** Admin gains access to election management tools.

Use Case 3: Add Candidate

- **Actor:** Admin
- **Description:** Admin enters the candidate's name and party affiliation, which triggers a blockchain transaction via the `addCandidate()` smart contract function.
- **Outcome:** Candidate is registered on the blockchain.

Use Case 4: Define Voting Dates

- **Actor:** Admin
- **Description:** Admin sets the start and end dates for the voting period using the `setDates()` smart contract function.
- **Outcome:** Voting is only active within the specified window.

Use Case 5: Cast Vote

- **Actor:** Voter
- **Description:** A voter selects a candidate and clicks the Vote button. A blockchain transaction is triggered via the `vote()` smart contract function.
- **Outcome:** Vote is recorded immutably on the blockchain; voter cannot vote again.

Use Case 6: View Results

- **Actor:** Voter / Admin
- **Description:** The voting page displays a real-time table with candidate names, parties, and vote counts retrieved from the blockchain.
- **Outcome:** Live election results are visible to all authenticated users.

Expected Outcomes

- A fully functional decentralized voting application accessible via a web browser.
- Votes recorded immutably on the Ethereum blockchain with no possibility of tampering.
- Role-based login system with JWT authentication.
- Admin-controlled election setup (candidates and voting window).
- Real-time vote count displayed on the voter dashboard.
- Demonstrated passing of all five key test cases (JWT auth, login, candidate registration, date setting, and vote casting).

Literature Survey

Overview

This section summarizes five key research papers reviewed during the development of this project. These papers informed the design decisions around security, transparency, scalability, and decentralization in blockchain-based e-voting systems.

Paper 1 — Online Voting System Using Blockchain

Authors: Vaibhav Anasune, Pradeep Choudhari, Madhura Kelapure, Pranali Shirke, and Prasad Halgaonkar

Summary: This paper surveys existing voting systems used by different countries and organizations, highlighting the security and transparency challenges with conventional, centralized approaches. The authors argue that advanced security methods are essential for effective online voting worldwide. In traditional electoral systems, the organization managing the central database has full control — creating a risk of manipulation. The paper makes the case for blockchain as a solution to these systemic issues.

Key Insight: Centralized database control is the root vulnerability in most existing electoral systems.

Paper 2 — A Systematic Literature Review and Meta-Analysis on Scalable Blockchain-Based Electronic Voting Systems

Authors: Uzma Jafar, Mohd Juzaiddin Ab Aziz, Zarina Shukur, and Hafiz Adnan Hussain

Summary: This systematic literature review (SLR) analyzed 76 English-language articles published between January 2017 and March 2022 from major academic databases. It identifies that while blockchain addresses authentication, data privacy, integrity, and transparency in e-voting, **scalability** has emerged as a fundamental barrier. The study evaluates various cryptographic solutions, performance parameters, and the advantages/disadvantages of different blockchain-based voting architectures.

Key Insight: Scalability is the most critical unsolved challenge in blockchain-based e-voting systems. Future solutions must consider cost, time, and throughput alongside security.

Paper 3 — A Survey of Blockchain Based on E-voting Systems

Authors: Yousif Osman Abuidris, Rajesh Kumar, and Wang Wenyong

Summary: This survey examines how blockchain's distributed, tamper-proof, and consensus-based architecture can be applied to create a secure digital platform for voting. It covers security and privacy concerns in existing e-voting systems and provides a comparative analysis of existing blockchain-based e-voting solutions. The authors also discuss applicable use cases in IoT and supply chain, noting that blockchain's core strengths translate well to voting.

Key Insight: A comparison of security and privacy requirements across existing systems reveals a gap in practical, production-ready implementations.

Paper 4 — Survey on Voting System Using Blockchain Technology

Authors: Mayur Shirsath, Mohit Zade, Riteshkumar Talke, Praful Wake, and Maya Shelke

Summary: This paper proposes an internet-based voting system supported by blockchain that solves accessibility problems in physical voting. Each vote is treated as a unique transaction in a private peer-to-peer blockchain. The application is designed to abstract technical complexity from users, making it accessible without requiring technical skills. The paper emphasizes that blockchain's encryption and hashing mechanisms ensure vote security, and notes that online voting significantly increases voter turnout.

Key Insight: Abstraction of blockchain complexity from the end-user is key to adoption — the system should be usable by voters with no technical background.

Paper 5 — A Survey on Smart Electronic Voting System Using Blockchain Technology

Authors: Naina Nagesh Dhepe and Dr. Pathan Mohd Shafi

Summary: With India being the world's largest democracy — over 668 million voters across 543 parliamentary constituencies — the need for a more secure and efficient voting mechanism is critical. This paper discusses security holes in current electronic voting machines (EVMs), particularly the inability to verify voter authenticity and protect machines from tampering. The proposed approach uses blockchain to develop a high-security, transparent voting machine, addressing both authentication and tamper-resistance.

Key Insight: Biometric integration and strong authentication are needed to confirm voter identity — a gap that blockchain alone cannot fully address.

Comparison of Existing Systems

Feature	Paper 1 (Anasune et al.)	Paper 2 (Jafar et al.)	Paper 3 (Abuidris et al.)	Paper 4 (Shirsath et al.)	Pap (Dhe Sh
Focus	Survey of existing systems	Scalability of blockchain voting	Security & privacy	Implementation & usability	Authen & tamp resistan
Blockchain Type	General	Various	Public/distributed	Private P2P	Blockch EVM
Scalability Addressed	❌	✅	Partial	❌	❌
Authentication Method	Not specified	Cryptographic	Not specified	Credential-based	Biomet (propo
Voter Privacy	✅	✅	✅	✅	✅
Real-time Results	❌	❌	❌	✅	❌
Practical Implementation	Survey only	Survey only	Survey only	✅	Partial

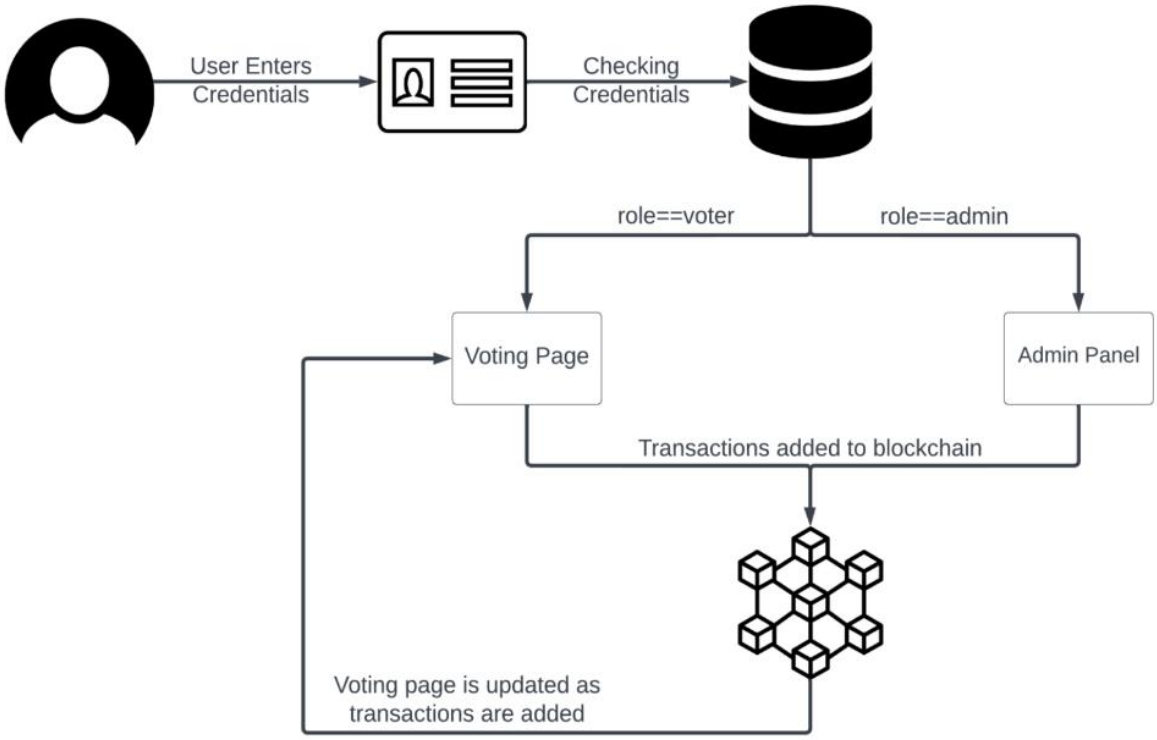
Research Gap Addressed by This Project

Based on the literature review, the following gaps were identified and addressed in this project:

- Most papers are surveys without a working implementation — **this project provides a fully functional system**.
- Scalability remains a challenge — **future scope includes Layer 2 solutions**.
- User complexity is a barrier — **this project abstracts blockchain interactions behind a simple web UI**.
- Authentication is often overlooked — **this project uses JWT + MySQL for secure, role-based login**.

System Architecture

Architecture Diagram



Architecture Overview

The system follows a **three-tier architecture** combining a web frontend, a Python API backend, and an Ethereum blockchain layer. Here is a high-level description of the flow:

1. The **user enters credentials** (Voter ID and password) on the Login Page.
2. Credentials are verified against a **MySQL database** via the FastAPI backend.
3. Based on the user's **role** stored in the database:
   - `role == voter` → redirected to the **Voting Page**
   - `role == admin` → redirected to the **Admin Panel**
4. The Admin can **add candidates** and **define voting dates** — these actions are sent as transactions to the Ethereum blockchain.
5. A Voter can **cast a vote** — this is also recorded as a blockchain transaction.
6. The **Voting Page updates in real time** as new transactions are added to the blockchain.

Component Breakdown

1.Frontend (HTML / JavaScript)

File	Responsibility
	Login form — collects Voter ID and password

`login.html`	
`index.html`	Voter dashboard — displays candidates, vote counts, and Vote button
`admin.html`	Admin dashboard — add candidates, define voting dates
`login.js`	Handles login API call and JWT token storage; redirects based on role
`App.js`	Connects to MetaMask, loads smart contract, handles voting and candidate display

2. Backend (Python FastAPI)

Component	Responsibility
`main.py`	FastAPI application entry point

Component	Responsibility
`/login` endpoint	Authenticates voter credentials against MySQL, returns JWT token and role
`authenticate()` middleware	Validates the JWT Bearer token on each protected request
`get_role()`	Queries MySQL to retrieve the user's role based on credentials
MySQL Database	Stores voter records with `voter_id`, `password`, and `role` fields

3. Blockchain Layer (Ethereum / Solidity)

Component	Responsibility
`Voting.sol`	Core smart contract — manages candidates, votes, and voting window
`Migrations.sol`	Truffle migration management contract
Ganache	Local Ethereum blockchain for development and testing
MetaMask	Browser wallet — signs transactions and connects to the Ethereum network
Web3.js	JavaScript library bridging the frontend and the smart contract
Truffle	Smart contract compilation, migration, and testing framework

Data Flow Summary

```
User (Browser)
    │
    ├──[Login Credentials]──► FastAPI Backend ──► MySQL DB (Verify User)
    │                                  │
    │                          JWT Token + Role returned
    │
    ├──[role == voter]──►  Voting Page (index.html)
    │                              │
    │                     App.js + Web3.js
    │                              │
    │                     MetaMask (signs tx)
    │                              │
    │                 Ethereum Blockchain (Ganache)
    │                     [Vote Transaction recorded]
    │                              │
    │                 Voting Page updates in real time
    │
    └──[role == admin]──►  Admin Page (admin.html)
```

```
                    |
          Add Candidate / Set Dates
                    |
     Ethereum Blockchain (Ganache)
     [Candidate/Date Tx recorded]
```

Module Descriptions

Voter Module

The voter module is designed for individuals eligible to participate in the voting process. Key features:

- **Secure Authentication** — voters log in using unique credentials (Voter ID + password).
- **Candidate Information** — voters can view candidate names, parties, and current vote totals.
- **Vote Casting** — voters select a candidate and submit their vote as a blockchain transaction.
- **Vote Verification** — voters can verify that their choice was accurately recorded on the blockchain.

Admin Module

The admin module is designed for election officials responsible for managing the voting system. Key features:

- **System Configuration** — admins define the start and end dates of the voting period.
- **Candidate Registration** — admins add candidates (name + party) to the smart contract.
- **Process Monitoring** — admins can verify candidate registration and confirm the election is running correctly.

Technologies Used

Programming Languages

Language	Version	Purpose
Solidity	0.5.16	Smart contract development on Ethereum
Python	3.9	Backend API development
JavaScript	ES6+	Frontend logic and blockchain interaction
HTML / CSS	HTML5	Web-based user interface

Blockchain Stack

Technology	Version	Purpose
Ethereum	—	Blockchain platform for deploying smart contracts
Solidity	0.5.16	Language for writing the `Voting.sol` smart contract
Truffle	5.7.6	Smart contract compilation, migration, and testing framework
Ganache	7.7.3	Local Ethereum blockchain for development and testing
MetaMask	Latest	Browser extension wallet for signing transactions and connecting to Ethereum
Web3.js	1.8.2	JavaScript library for interacting with the Ethereum blockchain from the browser

Backend Stack

Technology	Version	Purpose
Python	3.9	Backend runtime
FastAPI	Latest	High-performance async web framework for the REST API
MySQL	Port 3306	Relational database storing voter credentials and roles
PyJWT	Latest	JSON Web Token (JWT) generation and verification
mysql-connector-python	Latest	Python connector for MySQL database
python-dotenv	16.0.3	Environment variable management (secrets, DB credentials)

Frontend Stack

Technology	Version	Purpose
JavaScript	ES6+	Client-side logic
Web3.js	1.8.2	Interacts with the deployed smart contract via MetaMask

Technology	Version	Purpose
@truffle/contract	4.6.18	Provides a contract abstraction layer for cleaner Web3 interactions
Browserify	17.0.0	Bundles Node.js modules for browser use
jQuery	Latest	DOM manipulation and UI updates

Runtime & Package Management

Tool	Version	Purpose
Node.js	18.14.0	JavaScript runtime for frontend tooling
npm	Latest	Node package management
Express	4.18.2	Serves the static frontend files
jsonwebtoken	9.0.0	JWT handling on the Node.js side
dotenv	16.0.3	Environment configuration

Hardware Requirements

Component	Minimum Specification
Processor	2 GHz or higher
RAM	4 GB or more
Disk Space	100 GB or more
Internet Connection	Required for MetaMask and Ethereum node communication
Browser	Chrome or Firefox (MetaMask extension required)

Software Requirements Summary

```
Blockchain:
  - Ganache v7.7.3          (local Ethereum blockchain)
  - Truffle v5.7.6          (smart contract framework)
  - MetaMask                (browser wallet)
```

```
    - Solidity v0.5.16                    (smart contract language)

  Backend:

    - Python v3.9
    - FastAPI
    - MySQL (port 3306)

  Frontend:
    - Node.js v18.14.0
    - Web3.js v1.8.2

  Browser:
    - Google Chrome / Firefox with MetaMask extension
```

Methodology

System Working — Step by Step

Step 1: User Registration (Pre-configured)

Voter accounts (with `voter_id` , `password` , and `role`) are pre-registered in the MySQL database by the system administrator. Roles are either `admin` or `user` .

Step 2: Login & Authentication

1. User opens the Login Page and enters their **Voter ID** and **Password**.
2. The login form submits a `GET` request to the FastAPI backend at `/login` .
3. The backend's `authenticate()` middleware first validates that the Voter ID exists in the database.
4. The `get_role()` function then checks the password and retrieves the user's role.
5. A **JWT token** is generated using the secret key and returned to the frontend along with the role.
6. The frontend stores the token in `localStorage` and redirects the user based on their role:
   - `role == admin` → Admin Panel (`admin.html`)
   - `role == user` → Voting Page (`index.html`)

Step 3: Admin — Election Setup

1. Admin logs in and is presented with the Admin Panel.
2. Admin fills in a **candidate name** and **party** and clicks "Add Candidate."
3. This triggers the `addCandidate()` function in `Voting.sol` , which stores the candidate on the blockchain.
4. Admin also sets **voting start and end dates** using the `setDates()` function in the smart contract.

5. Dates can only be set once — the contract enforces this restriction.

Step 4: Voter — Casting a Vote

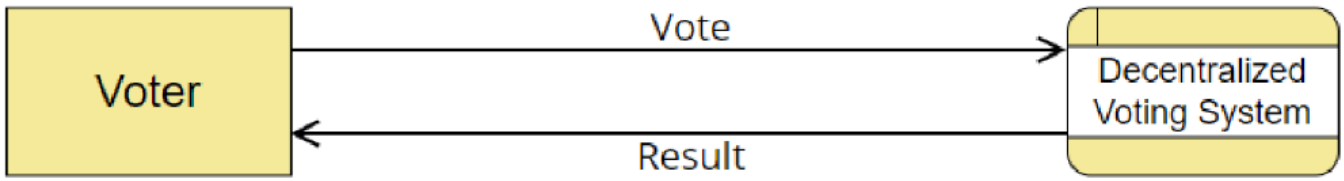
1. Voter logs in and lands on the Voting Page.
2. `App.js` connects to MetaMask, requests account access, and loads the deployed `Voting.sol` contract.
3. The list of candidates (name, party, vote count) is fetched from the blockchain and rendered in a table.
4. The voter selects a candidate using a radio button and clicks "Vote."
5. MetaMask prompts the user to confirm the transaction.
6. The `vote()` function in the smart contract verifies:
   - The current time is within the voting window (`votingStart <= now < votingEnd`).
   - The candidate ID is valid.
   - The voter's Ethereum address has **not voted before**.
7. The vote is recorded on the blockchain, and the page refreshes to reflect the updated vote count.

Step 5: Real-Time Results

- The Voting Page fetches candidate data from the blockchain on every load.
- Vote counts update in real time as transactions are confirmed.
- All users (voters and admin) can view live results.

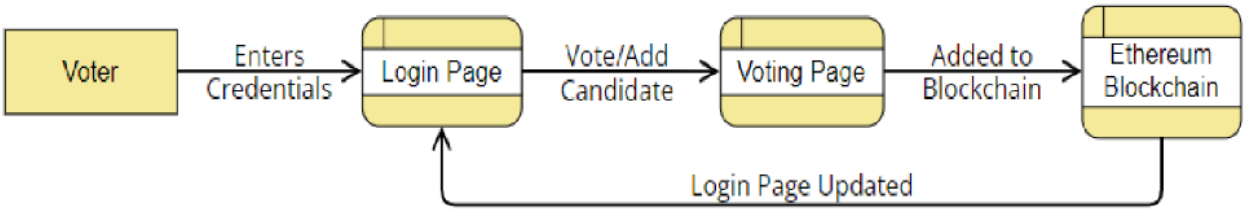
Data Flow Diagrams

Level 0 — Context Diagram



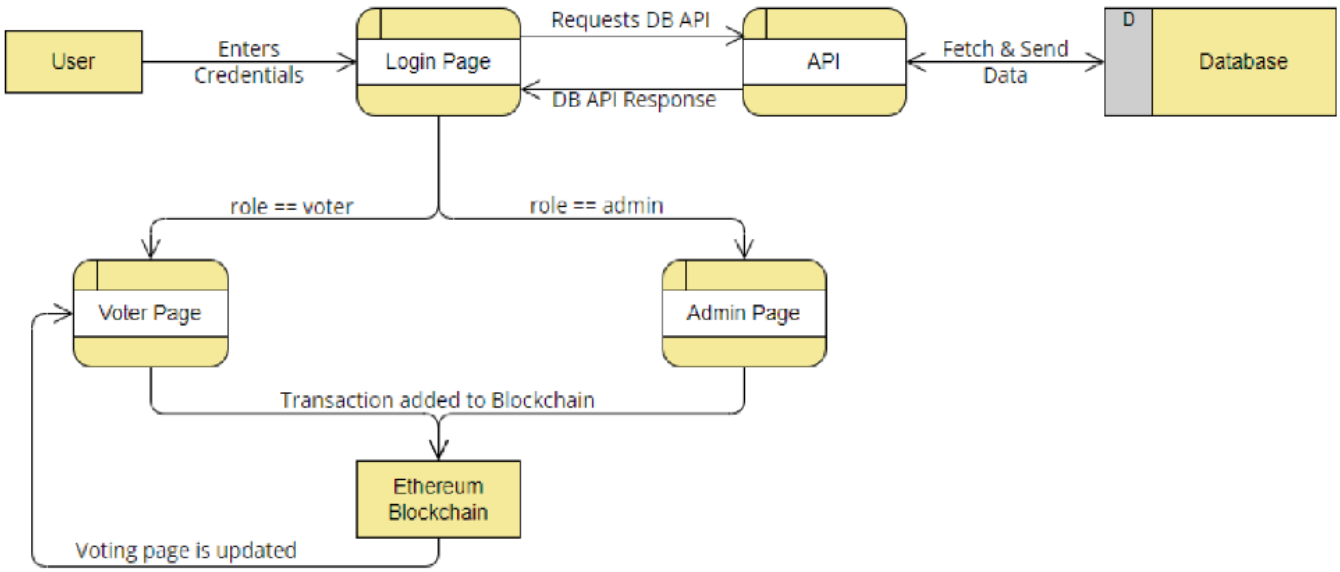
At the highest level, the **Voter** submits a **Vote** to the **Decentralized Voting System** and receives a **Result** in return.

Level 1 — System Decomposition



The voter enters credentials at the **Login Page**, which then redirects to the **Voting Page**. Votes (or candidate additions by admin) are added to the **Ethereum Blockchain**. The login page is updated with session information.

Level 2 — Detailed Flow

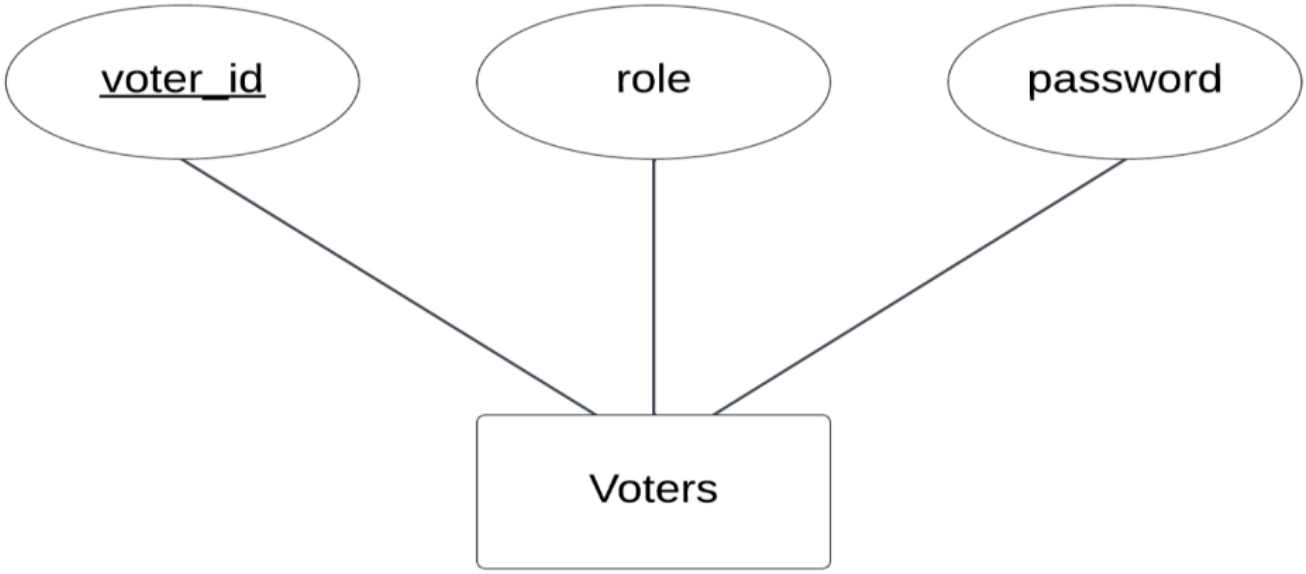


At this level, the system is decomposed further:

- **User → Login Page:** User enters credentials.
- **Login Page → API:** Requests database authentication.
- **API → Database:** Fetches voter record and role.
- **Role branching:**
  - `voter` → Voter Page (cast vote → Ethereum Blockchain → page updates)
  - `admin` → Admin Page (add candidate / set dates → Ethereum Blockchain)

ER Diagram

—



The database schema has a single `Voters` entity with three attributes:

- `voter_id` (Primary Key — underlined)
- `password`
- `role`

Smart Contract Logic Flow

```
Admin calls addCandidate(name, party)
    → countCandidates++
    → candidates[id] = Candidate(id, name, party, 0)

Admin calls setDates(startDate, endDate)
    → Requires: votingEnd == 0, votingStart == 0 (can only be set once)
    → Requires: endDate > startDate
    → Sets votingStart and votingEnd
```

```
Voter calls vote(candidateID)
    → Requires: votingStart <= now < votingEnd
    → Requires: 0 < candidateID <= countCandidates
    → Requires: voters[msg.sender] == false (not voted before)
    → Sets voters[msg.sender] = true
    → candidates[candidateID].voteCount++
```

Implementation

Repository

🔗 **GitHub Repository:** https://github.com/aaryansah7/E-Voting-System-using-Blockchain-Ganache-MetaMask-.git

Project Setup

Prerequisites

Before running the project, ensure the following are installed:

- Node.js v18.14.0
- Python 3.9
- Ganache v7.7.3
- Truffle v5.7.6 — `npm install -g truffle`
- MetaMask browser extension
- MySQL Server (port 3306)

Installation Steps

1. Clone the repository

```
git clone https://github.com/your-username/your-repo-name.git
cd your-repo-name
```

2. Install Node.js dependencies

```
npm install
```

3. Install Python dependencies

```
pip install fastapi uvicorn mysql-connector-python python-dotenv PyJWT
```

4. Configure environment variables

Create a `.env` file in the root directory:

```
MYSQL_USER=your_mysql_username
MYSQL_PASSWORD=your_mysql_password
MYSQL_HOST=localhost
MYSQL_DB=voting_db
SECRET_KEY=your_jwt_secret_key
```

5. Set up the MySQL database

```
CREATE DATABASE voting_db;
USE voting_db;

CREATE TABLE voters (
    voter_id VARCHAR(50) PRIMARY KEY,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin', 'user') NOT NULL
);

-- Insert sample voters
INSERT INTO voters VALUES ('admin01', 'adminpass', 'admin');
INSERT INTO voters VALUES ('voter01', 'voterpass', 'user');
```

6. Start Ganache

Open Ganache and start a local blockchain on port `7545` (or configure Truffle accordingly).

7. Compile and deploy the smart contract

```
truffle compile
truffle migrate --reset
```

8. Start the FastAPI backend

```
uvicorn main:app --reload --port 8000
```

9. Start the frontend server

```
node server.js
# Or use: npx serve -p 8080
```

10. Open the application

Navigate to `http://127.0.0.1:8080` in your browser. Ensure MetaMask is connected to the Ganache network (`http://127.0.0.1:7545`).

Folder Structure

```
decentralized-voting/
│
├── contracts/                  # Solidity smart contracts
│   ├── Migrations.sol
│   └── Voting.sol
│
├── migrations/                 # Truffle migration scripts
│   ├── 1_initial_migration.js
│   └── 2_deploy_contracts.js
│
├── build/contracts/            # Compiled contract ABIs (auto-generated)
│   └── Voting.json
│
├── src/                        # Frontend source files
│   ├── index.html              # Voter page
│   ├── admin.html              # Admin page
│   ├── login.html              # Login page
│   ├── js/
│   │   ├── App.js              # Web3 + smart contract interaction
│   │   └── login.js            # Login form handler
│   └── css/
│       └── styles.css
│
├── main.py                     # FastAPI backend
├── .env                        # Environment variables (not committed)
├── package.json                # Node.js dependencies
├── truffle-config.js           # Truffle configuration
└── requirements.txt            # Python dependencies
```

Key Code — Smart Contracts

`Migrations.sol`

```
pragma solidity ^0.5.15;

contract Migrations {
    address public owner;
    uint public last_completed_migration;

    modifier restricted() {
        require(msg.sender == owner, "Access restricted to owner");
        _;
    }

    constructor() public {
        owner = msg.sender;
    }

    function setCompleted(uint completed) public  restricted {
        last_completed_migration = completed;
    }

    function upgrade(address new_address) public restricted {
        Migrations upgraded = Migrations(new_address);
        upgraded.setCompleted(last_completed_migration);
    }
}
```

Voting.sol

```
pragma solidity ^0.5.15;

contract Voting {
    struct Candidate {
        uint id;
        string name;
        string party;
        uint voteCount;
    }

    mapping (uint => Candidate) public candidates;
    mapping (address => bool) public voters;
    uint public countCandidates;
    uint256 public votingEnd;
    uint256 public votingStart;

    function addCandidate(string memory  name, string memory party) public returns(uint) {
        countCandidates++;
        candidates[countCandidates]  = Candidate(countCandidates, name, party, 0);
        return countCandidates;
    }

    function vote(uint candidateID) public {
```

```
        require((votingStart <= now) && (votingEnd > now));
        require(candidateID > 0 && candidateID <= countCandidates);
        require(!voters[msg.sender]);
        voters[msg.sender] = true;
        candidates[candidateID].voteCount++;
    }

    function checkVote() public view returns(bool) {
        return voters[msg.sender];
    }

    function getCountCandidates() public view returns(uint) {
        return countCandidates;
    }

    function getCandidate(uint candidateID) public view returns(uint, string memory, string me
        return (
            candidateID,
            candidates[candidateID].name,
            candidates[candidateID].party,
            candidates[candidateID].voteCount
        );
    }

    function setDates(uint256 _startDate, uint256 _endDate) public {
        require((votingEnd == 0) && (votingStart == 0) && (_startDate + 1000000 > now) && (_en
        votingEnd = _endDate;
        votingStart = _startDate;
    }

    function getDates() public view returns (uint256, uint256) {
        return (votingStart, votingEnd);
    }
}
```

Key Code — Backend (Python FastAPI)

`main.py` — Login Endpoint

```
@app.get("/login")
async def login(request: Request, voter_id: str, password: str):
    await authenticate(request)
    role = await get_role(voter_id, password)
    token = jwt.encode(
        {'password': password, 'voter_id': voter_id, 'role': role},
        os.environ['SECRET_KEY'],
        algorithm='HS256'
```

```
    )
    return {'token': token, 'role': role}
```

`main.py` — Authentication Middleware

```
async def authenticate(request: Request):
    try:
        api_key = request.headers.get('authorization').replace("Bearer ", "")
        cursor.execute("SELECT * FROM voters WHERE voter_id = %s", (api_key,))
        if api_key not in [row[0] for row in cursor.fetchall()]:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Forbidden")
    except:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Forbidden")
```

Key Code — Frontend

`login.js` — Role-based Redirect

```
.then(data => {
    if (data.role === 'admin') {
        localStorage.setItem('jwtTokenAdmin', data.token);
        window.location.replace(`http://127.0.0.1:8080/admin.html?Authorization=Bearer ${local
    } else if (data.role === 'user') {
        localStorage.setItem('jwtTokenVoter', data.token);
        window.location.replace(`http://127.0.0.1:8080/index.html?Authorization=Bearer ${local
    }
})
```

`App.js` — Vote Function

```
vote: function() {
    var candidateID = $("input[name='candidate']:checked").val();
    if (!candidateID) {
        $("#msg").html("<p>Please vote for a candidate.</p>");
        return;
    }
    VotingContract.deployed().then(function(instance) {
        instance.vote(parseInt(candidateID)).then(function(result) {
            $("#voteButton").attr("disabled", true);
            $("#msg").html("<p>Voted</p>");
            window.location.reload(1);
        });
```

```
    });
}
```

Results

Testing Overview

The system was tested using **Unit Testing**, **Functional Testing**, and **Black/White Box Testing** methodologies. All five key test cases passed successfully.

Test Cases & Results

Test Case 1 — JWT Authorization

Field	Detail
Test Case No.	1
Test Type	Unit Test
Name	Checking JWT Authorization
Description	Verify that users cannot access the voting or admin page without a valid JWT token.
Input	Login request with Voter ID and Password
Expected Output	User should not be able to access protected pages without proper authorization.
Actual Output	User cannot access the voting or admin page without valid authorization.
Result	✅ **Pass**
Comments	Working properly.

Test Case 2 — Voter Login

Field	Detail
Test Case No.	2

Field	Detail
Test Type	Functional Test
Name	Verify User Login
Description	Verify that a registered user can log in to the voting portal using valid credentials.
Input	Valid `voter_id` and `password`
Expected Output	User logs in successfully if credentials match the database; unauthorized error shown otherwise.
Actual Output	User is able to log in with correct credentials only. Invalid credentials are rejected.
Result	✅ **Pass**
Comments	Working properly.

Test Case 3 — Candidate Registration

Field	Detail
Test Case No.	3
Test Type	Unit Test
Name	Verify Candidate Registration
Description	Verify that the admin can register a new candidate on the blockchain.
Input	Candidate name and party affiliation
Expected Output	Registration blockchain transaction should succeed and candidate should appear in the list.
Actual Output	Registration transaction is successful. Candidate appears on the voting page.
Result	✅ **Pass**
Comments	Working properly.

Test Case 4 — Voting Date Registration

Field	Detail
Test Case No.	4
Test Type	Unit Test
Name	Verify Date Registration
Description	Verify that the admin can define the start and end dates of the voting period via the smart contract.
Input	Start date and end date
Expected Output	Date transaction should succeed and the voting window should be set on the blockchain.
Actual Output	Date transaction is successful. Voting dates are displayed on the Voting Page.
Result	✅ **Pass**
Comments	Working properly.

Test Case 5 — Vote Casting

Field	Detail
Test Case No.	5
Test Type	Functional Test
Name	Verify Voting
Description	Verify that a registered voter can successfully cast a vote for a candidate.
Input	Select a candidate and click "Vote" button
Expected Output	Vote transaction should succeed; vote count for the selected candidate should increment.
Actual Output	Vote transaction is successful. Vote count updates in real time. Vote button is disabled after voting.
Result	✅ **Pass**
Comments	Working properly.

Test Results Summary

Test #	Test Type	Feature Tested	Result
1	Unit	JWT Authorization	✅ Pass
2	Functional	User Login	✅ Pass
3	Unit	Candidate Registration	✅ Pass
4	Unit	Voting Date Setup	✅ Pass
5	Functional	Vote Casting	✅ Pass

All 5 test cases passed. No failures recorded.

Key Observations

- The **Vote button is automatically disabled** after a voter has cast their vote, preventing double voting at the UI level (enforced at the contract level too).

- The **voting page updates in real time** after each transaction is confirmed on the local blockchain.
- The **JWT token is validated on every request** — unauthorized access returns a `401 Forbidden` response.
- **Candidate registration and date setting are one-time operations** — the smart contract enforces this restriction with `require` statements.
- **Metamask transaction confirmation** adds a UX step before any state change on the blockchain, ensuring voters consciously confirm their vote.

Challenges

Development Challenges

1. Blockchain Scalability

Problem: The Ethereum blockchain, especially on a public network, has limited transaction throughput (roughly 15–30 transactions per second on the mainnet). For a large-scale election with millions of voters, this creates a bottleneck where transactions could queue up or incur high gas fees.

Mitigation in this project: The system was tested on a local Ganache blockchain, which has no throughput limits. For production, Layer 2 scaling solutions (e.g., Polygon, Optimism) would be required.

2. Smart Contract Immutability

Problem: Once a smart contract is deployed on the blockchain, it cannot be modified. Any bugs in the `Voting.sol` contract logic — such as incorrect date validation or the `vote()` function — would require redeployment of an entirely new contract, losing existing data.

Mitigation in this project: Thorough unit testing was performed using Truffle before deployment. The `--reset` flag during migration was used during development to redeploy cleanly.

3. Cross-Origin Resource Sharing (CORS)

Problem: The frontend (running on port `8080`) needed to make API requests to the FastAPI backend (running on port `8000`). Browsers block cross-origin requests by default, causing login failures.

Solution: CORS middleware was explicitly configured in `main.py` to allow requests from `http://localhost:8080` and `http://127.0.0.1:8080` .

4. MetaMask and Web3 Integration

Problem: Connecting the browser frontend to the local Ganache blockchain required configuring MetaMask with a custom network and importing Ganache test accounts. Users without MetaMask installed would see a fallback warning and could not interact with the contract.

Mitigation: A fallback to `http://127.0.0.1:9545` was implemented in `App.js` , though this is not secure for production use.

5. Synchronizing On-Chain and Off-Chain Data

Problem: The system uses two separate data stores — MySQL (for voter credentials and roles) and the Ethereum blockchain (for votes and candidates). Keeping these in sync, especially after blockchain redeployment during development, required careful manual management.

Mitigation: The MySQL database only stores authentication data (which does not change with blockchain state), limiting the risk of data inconsistency.

6. Voter Authentication vs. Blockchain Anonymity

Problem: The system requires voters to authenticate using Voter ID and password (centralized), but votes themselves are linked to Ethereum wallet addresses (pseudonymous). Bridging these two worlds while maintaining privacy is a non-trivial design challenge.

Current approach: Authentication is handled off-chain (JWT + MySQL); the blockchain only records wallet-level interactions, not voter identities.

7. Gas Costs and Transaction Fees

Problem: Every write operation on the Ethereum blockchain (adding a candidate, setting dates, voting) costs gas. On the public mainnet, this could be prohibitively expensive for voters.

Mitigation: The project uses Ganache (local testnet) where gas is free. Production deployment would require either a sponsored gas model or a cost-efficient Layer 2 network.

System Limitations

Limitation	Description
Local Blockchain Only	The current implementation runs on Ganache — not a live public network.

Limitation	Description
No Biometric Authentication	Voter identity is verified only by Voter ID and password, which can be shared or stolen.
Single Admin	There is no multi-signature admin setup — a single admin account controls the entire election.
No Voter Registration UI	Voter accounts must be manually inserted into the MySQL database; there is no self-registration flow.
Date Set Once	Voting dates can only be set once via the smart contract — there is no mechanism to extend or postpone an election.
Browser Dependency	The application requires a MetaMask-enabled browser; it cannot function in environments without the extension.
No Candidate Removal	Once a candidate is added to the blockchain, they cannot be removed.

Future Scope

Planned Enhancements

1. Biometric Voter Authentication

Description: Integrate fingerprint or facial recognition technology to replace or supplement the current Voter ID + password system. This would ensure that only the legitimate voter can cast a ballot, eliminating the risk of credential sharing or identity theft.

Technologies to explore: WebAuthn API, fingerprint sensor integration, face recognition APIs.

2. Artificial Intelligence for Voter Insights

Description: Apply machine learning and data analytics to anonymized voting data to generate insights — such as voter turnout trends, participation by region or demographic, and anomaly detection (e.g., suspicious voting patterns that may indicate fraud).

Technologies to explore: Python (scikit-learn, pandas), data dashboards (Grafana, Power BI).

3. Real-Time Vote Counting Dashboard

Description: Build a more sophisticated real-time analytics dashboard with live charts and graphs that update as votes are recorded on the blockchain, offering a richer experience for observers and election officials.

Technologies to explore: React.js, Chart.js, WebSockets, The Graph (blockchain indexing protocol).

4. Deployment on a Public Ethereum Testnet / Mainnet

Description: Move from the local Ganache blockchain to a public Ethereum testnet (e.g., Sepolia or Goerli) or eventually the Ethereum mainnet for real-world testing and deployment.

Considerations: Gas costs, smart contract audit, wallet management for voters.

5. Layer 2 Scalability Solutions

Description: Implement the voting system on a Layer 2 network such as **Polygon**, **Optimism**, or **Arbitrum** to dramatically increase transaction throughput and reduce gas costs — making the system viable for national-scale elections.

6. Zero-Knowledge Proofs (ZKP) for Privacy

Description: Implement ZKP cryptographic techniques to allow voters to prove they are eligible and have voted without revealing their identity or vote choice. This would provide true anonymity while maintaining verifiability.

Technologies to explore: zk-SNARKs, Circom, SnarkJS.

7. Mobile Application

Description: Develop a native mobile app (iOS and Android) for the voting system, allowing voters to authenticate and cast votes from their smartphones without requiring a desktop browser with MetaMask.

Technologies to explore: React Native, WalletConnect (mobile wallet integration).

8. Multi-Signature Admin Control

Description: Replace the single admin account with a multi-signature scheme where multiple authorized officials must approve administrative actions (adding candidates, setting dates). This prevents unilateral manipulation by a single admin.

Technologies to explore: Gnosis Safe, multi-sig smart contract patterns.

9. Voter Self-Registration with KYC

Description: Allow voters to register themselves through the application with Know Your Customer (KYC) verification — identity documents verified against a government database — eliminating the need for manual database entry by administrators.

10.Inter-Planetary File System (IPFS) Integration

Description: Store candidate information and election metadata on IPFS (a decentralized file storage network) rather than relying on the blockchain for all data. This would reduce on-chain storage costs while maintaining decentralization.

Research Opportunities

Area	Research Question
Scalability	How can sharding or rollups be applied to blockchain voting without compromising security?
Privacy vs. Transparency	How can ZKP be used to balance voter anonymity with publicly verifiable results?
Governance	What governance mechanisms best suit decentralized election administration?
Usability	How can blockchain complexity be further abstracted for non-technical voters?
Cross-chain Voting	Can a single election span multiple blockchain networks?

Conclusion

Summary of Work

This project successfully designed and implemented a **Decentralized Voting System using Ethereum Blockchain** — a full-stack application that addresses the core limitations of traditional and centralized electronic voting systems.

The key components built and integrated include:

- A **Solidity smart contract** (`Voting.sol`) deployed on a local Ganache blockchain, handling candidate management, voting period enforcement, and tamper-proof vote recording.
- A **Python FastAPI backend** with JWT-based authentication and a MySQL database, providing secure role-based access for voters and administrators.
- A **JavaScript frontend** using Web3.js and the Truffle contract abstraction to connect users to the blockchain through MetaMask.
- A **two-role system** (Admin and Voter) with separate dashboards and responsibilities.
- Comprehensive **testing** across five test cases — all of which passed — covering JWT authorization, login, candidate registration, date configuration, and vote casting.

Key Learnings

- **Blockchain immutability** is a double-edged sword — it ensures vote integrity but requires careful contract design before deployment, as bugs cannot be patched post-deployment.
- **Smart contract security** requires rigorous testing; every `require` statement in Solidity acts as a critical safeguard against invalid operations.
- **Off-chain and on-chain data** must be carefully coordinated — this project demonstrated a clean separation between authentication (MySQL) and voting (Ethereum blockchain).
- **Web3 integration** adds complexity to frontend development, but libraries like Web3.js and Truffle significantly reduce the friction of interacting with smart contracts from the browser.
- **CORS, JWT, and API security** are just as important in blockchain applications as in traditional web apps — the blockchain layer alone is not sufficient for end-to-end security.

Final Outcome

The project demonstrates that blockchain technology offers a **viable, transparent, and tamper-proof alternative** to conventional voting systems. By eliminating the need for centralized intermediaries, it provides:

- ✅ Immutable vote records
- ✅ Real-time result visibility
- ✅ Remote voter participation
- ✅ Automated and auditable election processes
- ✅ Reduced cost compared to physical infrastructure

With continued enhancements — particularly in scalability (Layer 2), privacy (ZKP), and authentication (biometrics) — decentralized voting systems represent a significant step toward **building more**

democratic, accountable, and accessible electoral processes for the digital era.

> *"Decentralized Voting with Ethereum Blockchain offers a robust and transparent solution for secure elections. With continued enhancements, it has the potential to revolutionize the democratic process and empower citizens to participate in a trusted and efficient voting system."*

References

Research Papers

1. **Anasune, V., Choudhari, P., Kelapure, M., Shirke, P., & Halgaonkar, P.** "*Literature Survey on Online Voting System Using Blockchain.*" — Survey paper examining existing voting systems and the role of blockchain in improving security and transparency.
2. **Jafar, U., Ab Aziz, M. J., Shukur, Z., & Hussain, H. A.** "*A Systematic Literature Review and Meta-Analysis on Scalable Blockchain-Based Electronic Voting Systems.*" — Systematic review of 76 articles (2017–2022) on scalable blockchain e-voting solutions. Available at: IEEE Xplore *(search title for exact link)*
3. **Abuidris, Y. O., Kumar, R., & Wenyong, W.** "*A Survey of Blockchain Based on E-voting Systems.*" — Comparative analysis of security and privacy requirements in blockchain-based e-voting systems.
4. **Shirsath, M., Zade, M., Talke, R., Wake, P., & Shelke, M.** "*Survey on Voting System using Blockchain Technology.*" — Proposes a peer-to-peer private blockchain for secure internet-based voting with abstracted user experience.
5. **Dhepe, N. N., & Shafi, P. M.** "*A Survey on Smart Electronic Voting System Using Blockchain Technology.*" — Focuses on authentication and security improvements for EVM-based voting using blockchain in India's democratic context.

Official Documentation

Resource	Link
Ethereum Documentation	https://ethereum.org/en/developers/docs/
Solidity Language Docs	https://docs.soliditylang.org/en/v0.5.16/
Truffle Suite	https://trufflesuite.com/docs/

Resource	Link
Ganache	https://trufflesuite.com/ganache/
Web3.js Documentation	https://web3js.readthedocs.io/en/v1.8.2/
FastAPI Documentation	https://fastapi.tiangolo.com/
MetaMask Docs	https://docs.metamask.io/
PyJWT Documentation	https://pyjwt.readthedocs.io/
MySQL Connector/Python	https://dev.mysql.com/doc/connector-python/en/
Node.js	https://nodejs.org/en/docs/
@truffle/contract	https://github.com/trufflesuite/truffle/tree/develop/packages/contract

Additional References

- Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf
- Buterin, V. (2014). *Ethereum White Paper*. https://ethereum.org/en/whitepaper/

Demo

🎥 Video Demonstration

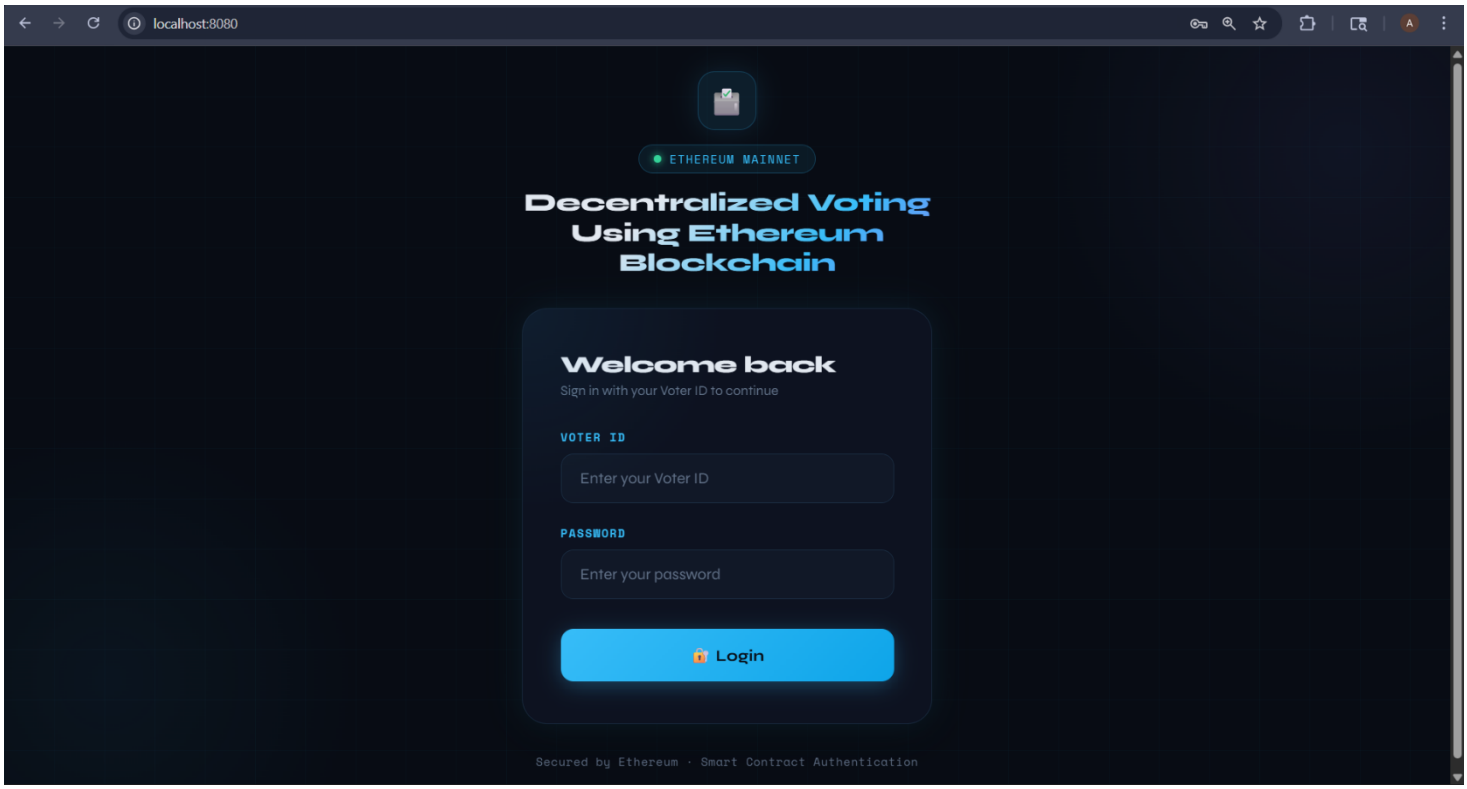
📌 **Video Demo: https://drive.google.com/file/d/1jRQPKpDoSE-6RK_iRYdJt3gltEDRc2yD/view?usp=sharing**

🖥 Live Application

> 🔗 **Deployed App (if applicable):** link-to-deployed-app
>
> If running locally, see the Implementation page for setup instructions.

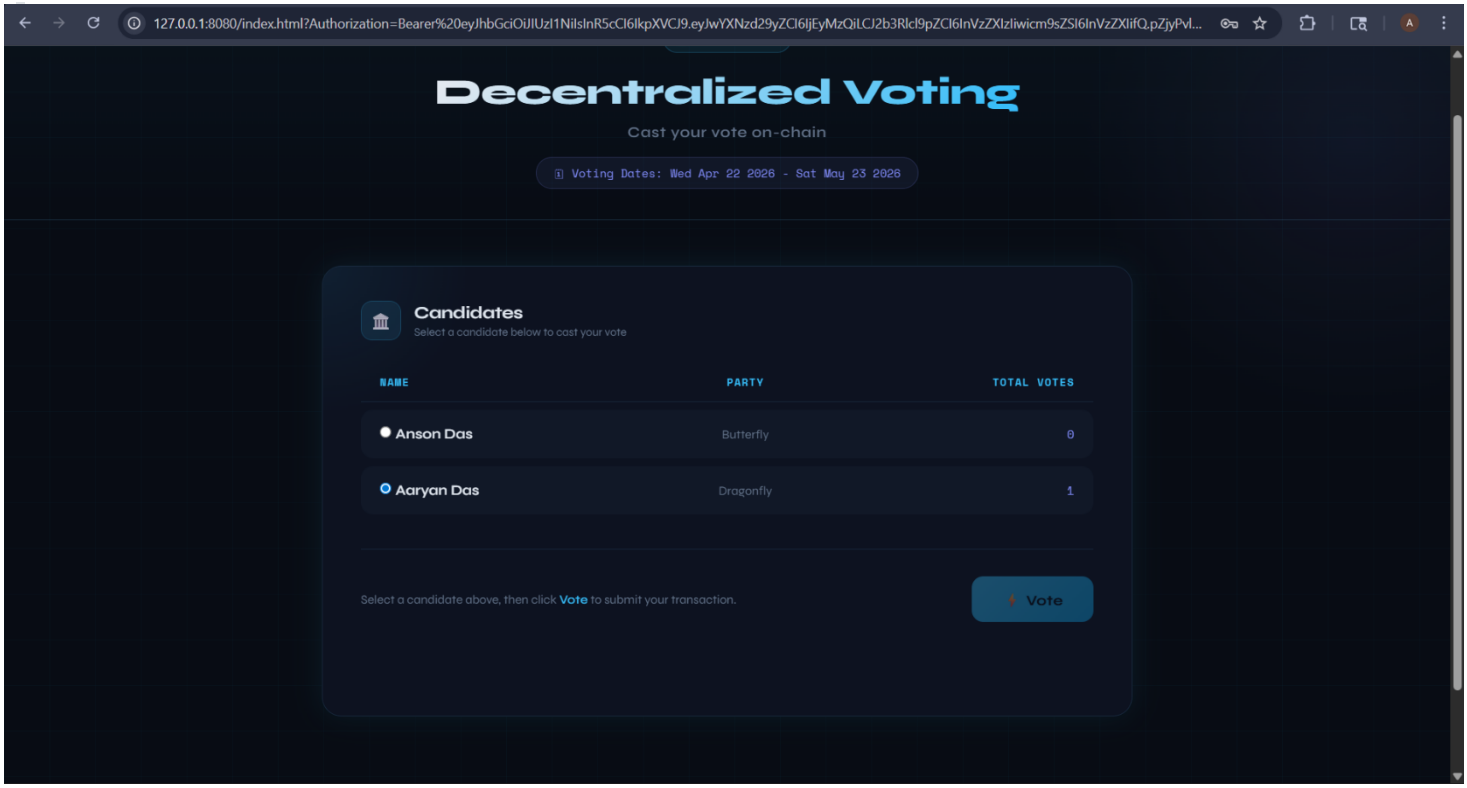
📸 Application Screenshots

1. Login Page



The login page accepts a **Voter ID** and **Password**. On submission, the credentials are validated against the MySQL database via the FastAPI backend. Users are redirected based on their role.

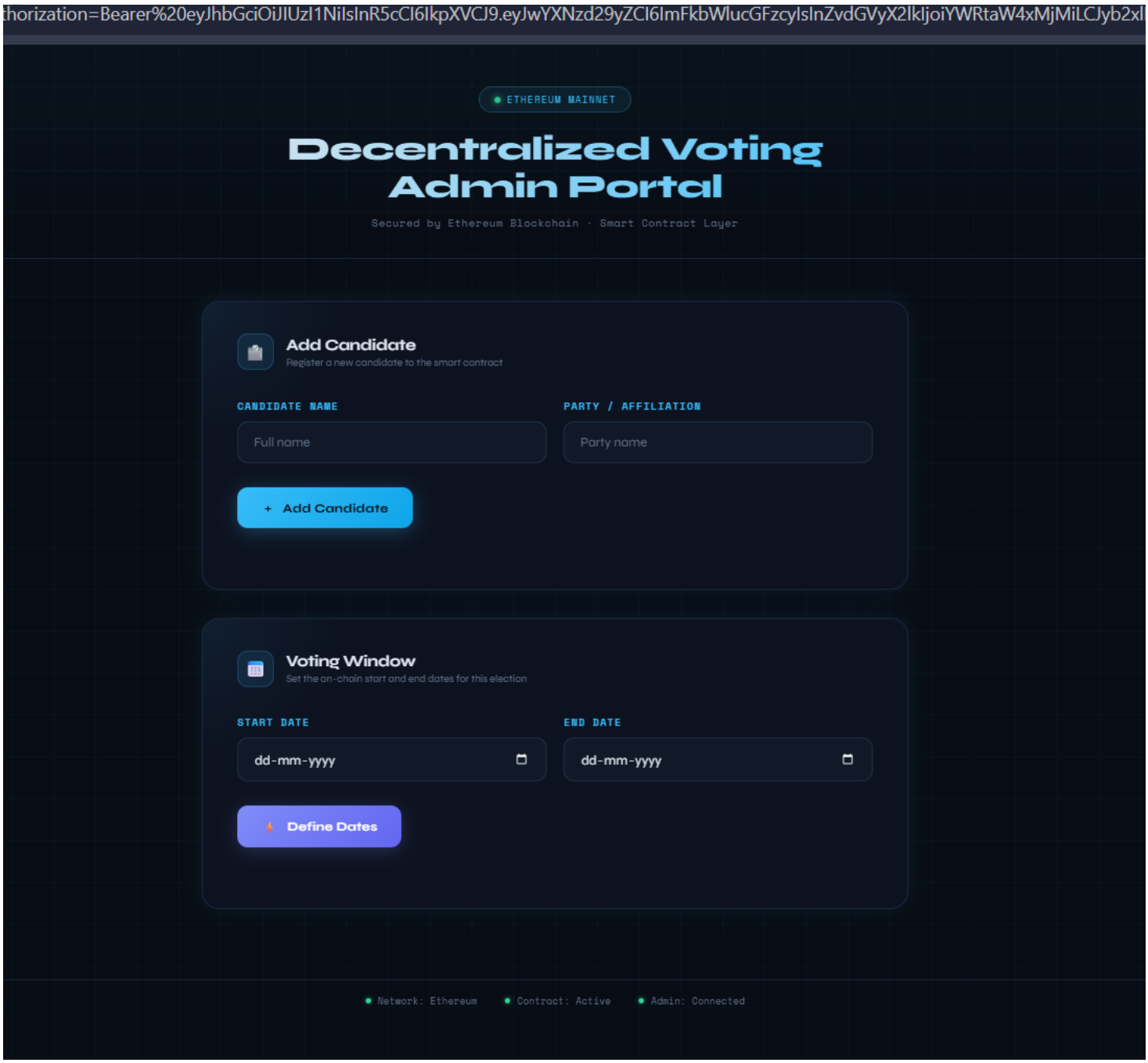
2. Voter Dashboard — Voting Page



The voter dashboard displays:

- The voting window dates (e.g., *Thu Jul 13 2023 - Fri Jul 14 2023*)
- A list of candidates with their party and live vote count
- A radio-button selection and a **Vote** button
- The connected MetaMask account address

3. Admin Panel



The admin dashboard provides:

- **Add Candidate** form — enter candidate Name and Party, then click "Add Candidate" to record on the blockchain.
- **Define Voting Dates** — set a Start Date and End Date for the election window.